

01_profiling_empresas_profesional_con_matching

March 29, 2026

1 Data Profiling e Integración de Fuentes para Clustering de Empresas

1.1 Objetivo

Este cuaderno documenta el proceso de análisis, validación e integración de dos fuentes de datos empresariales con el objetivo de construir un dataset adecuado para técnicas de clustering.

1.2 Fuentes de datos

- Dataset 1: Leads provenientes de CRM
- Dataset 2: Registro de horas trabajadas por empresa

1.3 Enfoque

Se sigue una metodología basada en:

1. Data profiling
2. Validación de hipótesis de integración
3. Evaluación de consistencia entre fuentes
4. Toma de decisiones basada en evidencia
5. Recomendación de siguiente paso (staging y estrategia de integración)

1.4 1. Imports y configuración

```
[1]: import pandas as pd
import numpy as np
import re
import unicodedata
from pathlib import Path

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
```

1.5 2. Configuración de archivos

```
[2]: LEADS_FILE = 'leads.xlsx'
HORAS_FILE = 'proyectos_empresa.xlsx'

N_FILAS_PRUEBA = None
```

1.6 3. Función de carga de datos

```
[3]: def leer_archivo(path, nrows=None):
    path = Path(path)
    suffix = path.suffix.lower()

    if suffix == '.csv':
        return pd.read_csv(path, nrows=nrows)
    elif suffix in ['.xlsx', '.xls']:
        return pd.read_excel(path, nrows=nrows)
    else:
        raise ValueError(f'Formato no soportado: {suffix}')
```

1.7 4. Carga de datasets

```
[4]: df_leads = leer_archivo(LEADS_FILE, nrows=N_FILAS_PRUEBA)
df_horas = leer_archivo(HORAS_FILE, nrows=N_FILAS_PRUEBA)

print('Leads:', df_leads.shape)
print('Horas:', df_horas.shape)
```

Leads: (440, 15)

Horas: (412, 18)

C:\Users\asus\AppData\Roaming\Python\Python312\site-packages\openpyxl\styles\stylesheet.py:237: UserWarning: Workbook contains no default style, apply openpyxl's default
warn("Workbook contains no default style, apply openpyxl's default")

1.8 4.b Perfilado rápido (calidad y esquema)

```
[5]: def _profile_table(df: pd.DataFrame) -> pd.DataFrame:
    out = pd.DataFrame({
        'columna': df.columns,
        'dtype': [str(df[c].dtype) for c in df.columns],
        'n_null': [int(df[c].isna().sum()) for c in df.columns],
        'pct_null': [float(df[c].isna().mean() * 100) for c in df.columns],
        'n_unique': [int(df[c].nunique(dropna=True)) for c in df.columns],
    })
    return out.sort_values(['pct_null', 'n_unique'], ascending=[False, True]).
    ↪reset_index(drop=True)
```

```

def _normalize_for_compare(value) -> str:
    if value is None:
        return ''
    if isinstance(value, float) and np.isnan(value):
        return ''
    s = str(value).strip()
    if not s:
        return ''
    s = unicodedata.normalize('NFKD', s)
    s = ''.join(ch for ch in s if not unicodedata.combining(ch))
    s = s.upper()
    s = re.sub(r'[^A-Z0-9 ]+', ' ', s)
    s = re.sub(r'\s+', ' ', s).strip()
    return s

def _company_quality(df: pd.DataFrame, col: str, label: str) -> None:
    if col not in df.columns:
        print(f'[{label}] Columna no encontrada: {col}')
        return
    s = df[col]
    s_str = s.astype('string')
    blank = s_str.isna() | (s_str.str.strip() == '')
    print(f'\n[{label}] Calidad de {col}:')
    print('- filas:', len(df))
    print('- n_null:', int(s.isna().sum()))
    print('- n_blank (null o vacío):', int(blank.sum()))
    print('- n_unique (no-null):', int(s.nunique(dropna=True)))
    print('- n_duplicadas (por valor no-null):', int(s.dropna().duplicated().
↪sum()))

    top = (s_str.fillna('')
           .map(lambda x: x.strip())
           .replace('', pd.NA)
           .dropna()
           .value_counts()
           .head(15)
           .reset_index())
    if len(top):
        top.columns = [col, 'freq']
        display(top)

    # Normalización rápida para estimar colisiones
    norm = s_str.map(_normalize_for_compare)
    norm = norm.replace('', pd.NA).dropna()
    if len(norm):
        print('- n_unique_norm:', int(norm.nunique()))

```

```

        print('- colisiones_por_norm (valores distintos que caen al mismo norm):
↳', int(norm.duplicated().sum()))

def _exact_match_summary() -> None:
    if 'Company' not in df_leads.columns or 'EMPRESA' not in df_horas.columns:
        print('\n[JOIN] No se puede calcular resumen: falta Company o EMPRESA.')
        return

    leads_raw = (df_leads['Company'].astype('string').fillna('').map(lambda x:
↳x.strip()))
    horas_raw = (df_horas['EMPRESA'].astype('string').fillna('').map(lambda x:
↳x.strip()))
    leads_raw = leads_raw.replace('', pd.NA).dropna()
    horas_raw = horas_raw.replace('', pd.NA).dropna()

    raw_matches = sorted(set(leads_raw).intersection(set(horas_raw)))
    print('\n[JOIN] Coincidencias exactas (sin normalizar):', len(raw_matches))
    if len(raw_matches):
        print(' Muestra:', raw_matches[:20])

    leads_norm = leads_raw.map(_normalize_for_compare).replace('', pd.NA).
↳dropna()
    horas_norm = horas_raw.map(_normalize_for_compare).replace('', pd.NA).
↳dropna()
    norm_matches = sorted(set(leads_norm).intersection(set(horas_norm)))
    print('[JOIN] Coincidencias exactas (normalizadas):', len(norm_matches))
    if len(norm_matches):
        print(' Muestra:', norm_matches[:20])

print('--- Perfilado LEADS ---')
display(_profile_table(df_leads))

print('\n--- Perfilado HORAS ---')
display(_profile_table(df_horas))

# Calidad de las llaves candidatas para join
_company_quality(df_leads, 'Company', 'LEADS')
_company_quality(df_horas, 'EMPRESA', 'HORAS')

# Evidencia cuantitativa: ¿hay intersección exacta?
_exact_match_summary()

```

--- Perfilado LEADS ---

	columna	dtype	n_null	pct_null	n_unique
0	First Name	float64	440	100.000000	0
1	Last Name	float64	440	100.000000	0
2	Email	float64	440	100.000000	0

3	Phone	float64	440	100.000000	0
4	Mobile	float64	440	100.000000	0
5	Website	float64	440	100.000000	0
6	No. of Employees	float64	440	100.000000	0
7	Annual Revenue	float64	440	100.000000	0
8	Linkedin	float64	440	100.000000	0
9	País.	object	415	94.318182	8
10	Lead Source	object	408	92.727273	1
11	Industry	object	408	92.727273	4
12	Cargo	object	87	19.772727	290
13	Interés	object	0	0.000000	1
14	Company	object	0	0.000000	330

--- Perfilado HORAS ---

	columna	dtype	n_null	pct_null	n_unique
0	AVANCE_REAL	float64	412	100.000000	0
1	AVANCE_ESTIMADO	float64	412	100.000000	0
2	ID_COL_RESPONSABLE	float64	379	91.990291	7
3	FACTURACION	float64	235	57.038835	120
4	HORAS_ESTIMADAS	float64	187	45.388350	107
5	HORAS_EJECUTADAS_FACTURABLES	float64	26	6.310680	314
6	HORAS_EJECUTADAS	float64	20	4.854369	319
7	FECHA_CORTE	datetime64[ns]	19	4.611650	5
8	EN_EJECUCION	float64	4	0.970874	2
9	MOSTRAR_LISTAS	int64	0	0.000000	2
10	LUGAR_POR_DEFECTO	int64	0	0.000000	2
11	TIPO_ALCANCE	object	0	0.000000	3
12	OCUPACION	object	0	0.000000	4
13	ID_ESP	int64	0	0.000000	15
14	EMPRESA	object	0	0.000000	120
15	ID_EMP	int64	0	0.000000	120
16	NOMBRE	object	0	0.000000	372
17	ID_PRO	int64	0	0.000000	412

[LEADS] Calidad de Company:

- filas: 440
- n_null: 0
- n_blank (null o vacío): 0
- n_unique (no-null): 330
- n_duplicadas (por valor no-null): 110

	Company	freq
0	GRUPO ALEN	10
1	PEPSICO	7
2	GRUPO BIMBO	7
3	WALMART	7
4	CASA CUERVO	5

5	BACARDI	5
6	SEGUROS MONTERREY NEW YORK LIFE	5
7	LAMOSA	5
8	UNILEVER	4
9	AIG	4
10	NOVARTIS	4
11	MABE	3
12	GRUPO CHEDRAUI	3
13	ASTRAZENECA	3
14	NATURESWEET	3

- n_unique_norm: 328
- colisiones_por_norm (valores distintos que caen al mismo norm): 112

[HORAS] Calidad de EMPRESA:

- filas: 412
- n_null: 0
- n_blank (null o vacío): 0
- n_unique (no-null): 120
- n_duplicadas (por valor no-null): 292

	EMPRESA	freq
0	Corporacion GPF	66
1	Corporación Maresa	22
2	FPA	21
3	Veolia Latam	21
4	Aseguradora del Sur	20
5	Intaco	12
6	La Fabril	10
7	Telefónica EC	10
8	NOVA Ecuador	10
9	Millicom - Telefónica PA, NI	9
10	Adium	8
11	Veolia China	7
12	Seguros del Pichincha	7
13	Veolia Argentina	6
14	Induglob	6

- n_unique_norm: 119
- colisiones_por_norm (valores distintos que caen al mismo norm): 293

[JOIN] Coincidencias exactas (sin normalizar): 0

[JOIN] Coincidencias exactas (normalizadas): 0

1.9 5. Validación de hipótesis de integración

1.9.1 Hipótesis

Se plantea que:

Las empresas presentes en el dataset de leads deberían coincidir con las empresas presentes en el dataset de horas trabajadas.

1.9.2 Objetivo

Validar si es posible realizar un join confiable entre ambas fuentes.

1.10 6. Evaluación detallada (coincidencias exactas normalizadas y aproximadas)

El resumen de la sección 4.b (Perfilado rápido) puede ampliarse con una rutina más detallada de normalización y similitud. Esta sección se conserva porque aporta evidencia técnica adicional:

- normaliza nombres con eliminación de acentos,
- reduce ruido por sufijos legales comunes,
- calcula coincidencias exactas después de normalizar,
- y genera sugerencias por similitud para evidenciar que no existen matches confiables.

Esto respalda formalmente la decisión de **no forzar un join** entre ambas fuentes usando solo el nombre de la empresa.

```
[28]: import re
import unicodedata
import pandas as pd

def _strip_accents(s: str) -> str:
    return ''.join(ch for ch in unicodedata.normalize('NFKD', s) if not
↳ unicodedata.combining(ch))

def _normalize_name(s: str) -> str:
    if s is None:
        return ""
    s = str(s).strip()
    s = _strip_accents(s)
    s = s.upper()
    # quitar caracteres raros, dejar letras/números/espacios
    s = re.sub(r"[^A-Z0-9 ]+", " ", s)
    # quitar sufijos legales comunes
    s = re.sub(r"\b(SA|S A|S\.A|S\.A\.
↳ S|SAS|LTDA|CIA|CORP|INC|LLC|DE|DEL|LA|EL)\b", " ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s

# Tomar listas directamente desde los DataFrames (evita dependencia de
↳ secciones eliminadas)
if 'Company' not in df_leads.columns or 'EMPRESA' not in df_horas.columns:
    print("No se puede comparar: falta Company o EMPRESA en los dataframes.")
else:
    companies = (df_leads['Company'].astype('string').fillna(''))
```

```

        .map(lambda x: x.strip())
        .replace('', pd.NA)
        .dropna()
        .unique()
        .tolist()
empresas = (df_horas['EMPRESA'].astype('string').fillna('')
            .map(lambda x: x.strip())
            .replace('', pd.NA)
            .dropna()
            .unique()
            .tolist())

if not companies or not empresas:
    print("No hay valores suficientes para comparar (listas vacías).")
else:
    comp_norm = {c: _normalize_name(c) for c in companies}
    emp_norm = {e: _normalize_name(e) for e in empresas}

    # 1) Coincidencias exactas después de normalizar
    inv_emp = {}
    for e, ne in emp_norm.items():
        inv_emp.setdefault(ne, []).append(e)

    exact_matches = []
    for c, nc in comp_norm.items():
        for e in inv_emp.get(nc, []):
            exact_matches.append({
                "Company": c,
                "EMPRESA": e,
                "tipo": "exact_norm",
                "score": 100
            })

    df_exact = pd.DataFrame(exact_matches)
    print(f"Coincidencias EXACTAS (tras normalizar): {len(df_exact)}")
    if len(df_exact):
        display(df_exact.sort_values(['Company', 'EMPRESA']).
        ↪reset_index(drop=True))

    # 2) Coincidencias por similitud (fuzzy). Si está rapidfuzz, mejor; si
    ↪no, usa difflib.
    rows = []
    try:
        from rapidfuzz import process, fuzz
        scorer = fuzz.token_set_ratio
        emp_choices = list(emp_norm.keys())
        for c, nc in comp_norm.items():

```



```

        best = process.extractOne(nc, emp_choices, scorer=scorer)
        if best is None:
            continue
        best_norm, score, _ = best
        e_orig = inv_emp.get(best_norm, [None])[0]
        rows.append({
            "Company": c,
            "Company_norm": nc,
            "EMPRESA_best": e_orig,
            "EMPRESA_norm": best_norm,
            "score": float(score),
        })
    engine = "rapidfuzz"
except Exception:
    from difflib import SequenceMatcher

    def ratio(a, b):
        return SequenceMatcher(None, a, b).ratio() * 100

    emp_choices = list(emp_norm.keys())
    for c, nc in comp_norm.items():
        best_norm = None
        best_score = -1
        for en in emp_choices:
            sc = ratio(nc, en)
            if sc > best_score:
                best_score = sc
                best_norm = en
        e_orig = inv_emp.get(best_norm, [None])[0] if best_norm is not None
        ↪None else None
        rows.append({
            "Company": c,
            "Company_norm": nc,
            "EMPRESA_best": e_orig,
            "EMPRESA_norm": best_norm,
            "score": float(best_score),
        })
    engine = "difflib"

df_fuzzy = pd.DataFrame(rows).sort_values('score', ascending=False).
↪reset_index(drop=True)
print(f"\nMotor de similitud: {engine}")
print("Sugerencia: revisar coincidencias con score alto (p.ej. >= 90).")

display(df_fuzzy.head(30))
umbral = 90
df_high = df_fuzzy[df_fuzzy['score'] >= umbral]

```

```
print(f"\nCoincidencias sugeridas con score >= {umbral}:\n")
↳len(df_high)})")
display(df_high.reset_index(drop=True))
```

Coincidencias EXACTAS (tras normalizar): 0

Motor de similitud: difflib

Sugerencia: revisar coincidencias con score alto (p.ej. >= 90).

	Company	Company_norm	EMPRESA_best	EMPRESA_norm	score
0	OSRAM	OSRAM	CORSAM	CORSAM	72.727273
1	FEMSA	FEMSA	FADESA	FADESA	72.727273
2	SMI	SMI	BMI	BMI	66.666667
3	UCB	UCB	UPC	UPC	66.666667
4	BIC	BIC	BAC	BAC	66.666667
5	Chronos	CHRONOS	CONSEP	CONSEP	61.538462
6	BACARDI	BACARDI	BANRED	BANRED	61.538462
7	LACOSTE	LACOSTE	CONSEP	CONSEP	61.538462
8	Onapis	ONAPIS	AVIS	AVIS	60.000000
9	PEPSICO	PEPSICO	SIC	SIC	60.000000
10	Abbvie	ABBVIE	AVIS	AVIS	60.000000
11	BACHOCO	BACHOCO	BAC	BAC	60.000000
12	BALL	BALL	BAC	BAC	57.142857
13	PROESSA	PROESSA	PROPHAR	PROPHAR	57.142857
14	PPG	PPG	KPMG	KPMG	57.142857
15	URBI	URBI	BMI	BMI	57.142857
16	AIG	AIG	AVIS	AVIS	57.142857
17	ICONN	ICONN	CONSEP	CONSEP	54.545455
18	Pozuelo	POZUELO	PUCE	PUCE	54.545455
19	Arcor	ARCOR	CORSAM	CORSAM	54.545455
20	Parauco	PARAUCO	PUCE	PUCE	54.545455
21	COMEX	COMEX	CONSEP	CONSEP	54.545455
22	ESACERO S.A.	ESACERO	ESPE	ESPE	54.545455
23	TRANE	TRANE	BANRED	BANRED	54.545455
24	ONTEX	ONTEX	CONSEP	CONSEP	54.545455
25	CXO Forum	CXO FORUM	CORSAM	CORSAM	53.333333
26	Confiteca	CONFITECA	CONSEP	CONSEP	53.333333
27	NOVAMEX	NOVAMEX	None	NOVA Ecuador	52.631579
28	ALCIONE MX	ALCIONE MX	CONSEP	CONSEP	50.000000
29	LAMOSA	LAMOSA	FADESA	FADESA	50.000000

Coincidencias sugeridas con score >= 90: 0

Empty DataFrame

Columns: [Company, Company_norm, EMPRESA_best, EMPRESA_norm, score]

Index: []

1.10.1 Interpretación técnica de esta validación

Si esta sección produce:

- **0 coincidencias exactas tras normalización**, y
- **0 coincidencias con score alto**,

entonces existe evidencia suficiente para afirmar que ambas fuentes no pueden integrarse de forma confiable mediante el nombre de la empresa.

Este resultado no representa una falla del código, sino un hallazgo del análisis de calidad e integración de datos.

Nota de organización: la generación de **data cruda normalizada** (staging) y el **export** a CSV se movieron a `02_data_cleaning/01_raw_normalizado_export.ipynb` para no mezclar ingesta/profiling con cleaning.

1.11 7. Resultado de la validación

No se encontraron coincidencias exactas entre las empresas de ambas fuentes.

1.11.1 Interpretación

Esto indica que:

- Las fuentes no están alineadas
- No es posible realizar un join directo confiable
- Existe inconsistencia en la representación de entidades

1.11.2 Conclusión

Se rechaza la hipótesis de integración directa.

1.12 8. Próximos pasos (según evidencia)

- No forzar join `Company EMPRESA` por nombre: el profiling muestra que no hay coincidencias confiables.
- Preparar staging normalizado para BD/joins posteriores en `02_data_cleaning/01_raw_normalizado_export.ipynb` (export a CSV).
- Definir estrategia alternativa de integración (p.ej. catálogo unificado de empresas / matching asistido / llaves adicionales).

1.13 9. Universo unificado de empresas (LEADS HORAS) y verificación contra SCVS

Objetivo: como no hay coincidencias exactas entre ambas fuentes, construimos el **universo unificado** de empresas (a partir de ambas) y evaluamos cobertura en una fuente externa (SCVS / Superintendencia de Compañías).

- Esta sección **no exporta** CSV; solo genera dataframes y métricas para el análisis.

```
[5]: # --- (A) Universo unificado (union all) de empresas normalizadas ---  
def _normalize_text_for_matching_local(value) -> str:
```

```

    # Reutiliza el estilo de normalización usado en el profiling (sin reglas de
    ↪negocio).
    try:
        return _normalize_for_compare(value) # definido en la sección 4.b
    except NameError:
        # fallback defensivo si se ejecuta esta celda en aislamiento
        if value is None:
            return ''
        if isinstance(value, float) and np.isnan(value):
            return ''
        s = str(value).strip()
        if not s:
            return ''
        s = unicodedata.normalize('NFKD', s)
        s = ''.join(ch for ch in s if not unicodedata.combining(ch))
        s = s.upper()
        s = re.sub(r'[A-Z0-9 ]+', ' ', s)
        s = re.sub(r'\s+', ' ', s).strip()
        return s

def _normalize_colkey(name) -> str:
    s = '' if name is None else str(name)
    s = unicodedata.normalize('NFKD', s)
    s = ''.join(ch for ch in s if not unicodedata.combining(ch))
    s = s.lower().strip()
    s = re.sub(r'[a-z0-9]+', '_', s)
    s = re.sub(r'_+', '_', s).strip('_')
    return s

def _find_col_by_candidates(df: pd.DataFrame, candidates: list[str]) -> str |
    ↪None:
    norm_to_actual = {_normalize_colkey(c): c for c in df.columns}
    for cand in candidates:
        k = _normalize_colkey(cand)
        if k in norm_to_actual:
            return norm_to_actual[k]
    return None

def _prep_distinct_series(df: pd.DataFrame, col: str | None) -> pd.Series:
    if not col or col not in df.columns:
        return pd.Series(dtype='string')
    s = df[col].dropna().astype('string')
    s = s.str.strip()
    s = s[s != '']
    s = s.map(_normalize_text_for_matching_local)
    s = s.replace('', pd.NA).dropna()
    # distinct dentro de cada fuente

```

```

return s.drop_duplicates()

# Detectar columna de empresa en cada fuente (robusto a headers)
leads_company_col = _find_col_by_candidates(df_leads, ['Company', 'EMPRESA', '↵
↳ 'Empresa', 'Razon Social', 'Razón Social', 'Nombre Empresa'])
horas_company_col = _find_col_by_candidates(df_horas, ['EMPRESA', 'Empresa', '↵
↳ 'Company', 'CLIENTE', 'Cliente', 'Razon Social', 'Razón Social', 'Nombre_
↳ Empresa'])

leads_empresas = _prep_distinct_series(df_leads, leads_company_col)
horas_empresas = _prep_distinct_series(df_horas, horas_company_col)

# union all (concat) luego de distinct por separado
empresas_union = pd.concat([horas_empresas, leads_empresas], ignore_index=True)

# Para tener un solo conjunto final, deduplicar globalmente
empresas_union = empresas_union.drop_duplicates().reset_index(drop=True)

df_empresas_union_all = pd.DataFrame({'empresa_norm': empresas_union})

print('Columna LEADS detectada:', leads_company_col)
print('Columna HORAS detectada:', horas_company_col)
print('df_empresas_union_all:', df_empresas_union_all.shape)
display(df_empresas_union_all.head(30))

```

```

Columna LEADS detectada: Company
Columna HORAS detectada: EMPRESA
df_empresas_union_all: (447, 1)

```

```

      empresa_norm
0          3DPHARMA
1             AVIS
2            ADIUM
3          ALMEXA
4    ALPER SEGUROS
5          ARAUCO
6  ASEGURADORA DEL SUR
7  ASESORIA Y CONTROL
8        AUTOSHARE
9             BAC
10          BANRED
11            BMI
12    BANCO FINANCIERO
13    BANCO DEL AUSTRO
14          BEKAERT
15          BEMIS
16          BITSO
17    BLOOMIN BRANDS

```

```

18         BOSTON BSCI
19         CEPESA
20         CONSEP
21         CORSAM
22         CETI VEHICULOS
23         CHARGERLOGISTICS
24         CLINTON FOUNDATION
25         COLSUBSIDIO
26         COMANDATO
27 COMESTIBLES INTEGRALES
28         CORONA
29         CORPORACION GPF

```

```

[19]: # --- (B) SCVS: cargar archivos descargados y calcular cobertura (salida
      ↪limpia) ---
import warnings
warnings.filterwarnings('ignore', message='Workbook contains no default style*')

SCVS_DIR = Path('data_super_compañías')
if not SCVS_DIR.exists():
    SCVS_DIR = Path('..') / '01_data_ingestion_enrichment' /
    ↪'data_super_compañías'

# Ajustes
MATCH_THRESHOLD = 85 # prueba 80/85/90 según tolerancia
VERBOSE = False      # True si quieres ver samples/tablas

def _read_excel_sheet_names(path: Path) -> list[str]:
    xls = pd.ExcelFile(path)
    return list(xls.sheet_names)

def _read_csv_best_effort(path: Path) -> pd.DataFrame | None:
    encodings = ['utf-8-sig', 'utf-8', 'latin-1', 'cp1252']
    seps = [',', ';', '\t', '|']
    for enc in encodings:
        for sep in seps:
            try:
                df = pd.read_csv(path, encoding=enc, sep=sep)
                if df is not None and not df.empty and df.shape[1] >= 2:
                    return df
            except Exception:
                continue
    for enc in encodings:
        try:
            df = pd.read_csv(path, encoding=enc, sep=None, engine='python')
            if df is not None and not df.empty and df.shape[1] >= 2:
                return df

```

```

        except Exception:
            continue
    return None

def _coerce_ruc(value):
    if value is None:
        return pd.NA
    if isinstance(value, float) and np.isnan(value):
        return pd.NA
    s = str(value).strip()
    if not s:
        return pd.NA
    s = re.sub(r'\D+', '', s)
    return s if s else pd.NA

def _is_ruc_like(series: pd.Series) -> float:
    s = series.astype('string').fillna('')
    s = s.str.replace(r'\.0$', '', regex=True)
    d = s.str.replace(r'\D+', '', regex=True)
    ok = d.str.len().between(10, 13)
    return float(ok.mean())

def _is_name_like(series: pd.Series) -> float:
    s = series.astype('string').fillna('')
    s = s.map(lambda x: x.strip())
    s = s[s != '']
    if len(s) == 0:
        return 0.0
    has_letters = s.str.contains(r'[A-Za-zÁÉÍÓÚŨÑáéíóúũñ]', regex=True)
    avg_len = s.str.len().mean()
    return float(has_letters.mean()) * float(min(avg_len / 30.0, 1.0))

def _extract_scvs_from_df(df: pd.DataFrame, fuente_archivo: str, fuente_hoja:
↳str | None = None) -> pd.DataFrame | None:
    if df is None or df.empty:
        return None

    name_candidates = [
        'razon social', 'razón social', 'razon_social', 'nombre', 'nombre_
↳compania', 'nombre compañía',
        'denominacion', 'denominación', 'compania', 'compañia', 'institucion',
↳'institución', 'empresa', 'entidad'
    ]
    ruc_candidates = [
        'ruc', 'identificacion', 'identificación', 'identificacion tributaria',
↳'identificación tributaria',

```

```

        'numero identificacion', 'número identificación',
↪ 'numero_de_identificacion', 'nro_identificacion',
        'documento', 'numero_documento', 'nro_documento'
    ]
    employees_candidates = ['empleados', 'n_empleados', 'numero empleados',
↪ 'número empleados', 'personal']

    col_name = _find_col_by_candidates(df, name_candidates)
    col_ruc = _find_col_by_candidates(df, ruc_candidates)
    col_emp = _find_col_by_candidates(df, employees_candidates)

    if col_name is not None:
        tmp = pd.DataFrame({
            'nombre_oficial_raw': df[col_name],
            'ruc': (df[col_ruc] if col_ruc is not None else pd.NA),
            'n_empleados': (df[col_emp] if col_emp is not None else pd.NA),
        })
    else:
        ruc_col = None
        best_ruc = 0.0
        for c in df.columns:
            score = _is_ruc_like(df[c])
            if score > best_ruc:
                best_ruc = score
                ruc_col = c
        name_col = None
        best_name = 0.0
        for c in df.columns:
            if c == ruc_col:
                continue
            score = _is_name_like(df[c])
            if score > best_name:
                best_name = score
                name_col = c
        if name_col is None and ruc_col is None:
            return None
        tmp = pd.DataFrame({
            'nombre_oficial_raw': (df[name_col] if name_col is not None else pd.
↪ NA),
            'ruc': (df[ruc_col] if ruc_col is not None else pd.NA),
            'n_empleados': pd.NA,
        })

    tmp['ruc'] = tmp['ruc'].map(_coerce_ruc)
    tmp['nombre_oficial_raw'] = tmp['nombre_oficial_raw'].astype('string')
    tmp['nombre_oficial_raw'] = tmp['nombre_oficial_raw'].fillna('').map(lambda
↪ x: x.strip())

```



```

    tmp = tmp[tmp['nombre_oficial_raw'] != '']
    tmp['nombre_oficial_norm'] = tmp['nombre_oficial_raw'].
↳map(_normalize_text_for_matching_local)
    tmp = tmp[tmp['nombre_oficial_norm'].astype('string').str.strip() != '']
    tmp['fuente_archivo'] = fuente_archivo
    tmp['fuente_hoja'] = '' if fuente_hoja is None else str(fuente_hoja)
    return tmp.reset_index(drop=True)

def _extract_scvs_from_excel_sheet(path: Path, sheet_name: str) -> pd.DataFrame:
    ↳| None:
        try:
            df = pd.read_excel(path, sheet_name=sheet_name)
            part = _extract_scvs_from_df(df, fuente_archivo=path.name,
↳fuente_hoja=sheet_name)
            if part is not None and len(part):
                return part
        except Exception:
            pass
        try:
            raw = pd.read_excel(path, sheet_name=sheet_name, header=None)
            part = _extract_scvs_from_df(raw, fuente_archivo=path.name,
↳fuente_hoja=sheet_name)
            return part
        except Exception:
            return None

# 1) Cargar SCVS (xlsx + csv)
scvs_excels = sorted([p for p in SCVS_DIR.glob('*.xlsx') if p.is_file()])
scvs_csvs = sorted([p for p in SCVS_DIR.glob('*.csv') if p.is_file()])

scvs_parts: list[pd.DataFrame] = []
for path in scvs_excels:
    for sheet_name in _read_excel_sheet_names(path):
        part = _extract_scvs_from_excel_sheet(path, sheet_name)
        if part is not None and len(part):
            scvs_parts.append(part)

for path in scvs_csvs:
    df = _read_csv_best_effort(path)
    part = _extract_scvs_from_df(df, fuente_archivo=path.name, fuente_hoja=None)
    if part is not None and len(part):
        scvs_parts.append(part)

if not scvs_parts:
    raise RuntimeError('No se pudo extraer ninguna tabla SCVS desde los
↳archivos en la carpeta.')

```

```

df_scvs_master = pd.concat(scvs_parts, ignore_index=True)
df_scvs_master = df_scvs_master.dropna(subset=['nombre_oficial_norm'])
df_scvs_master = df_scvs_master.drop_duplicates(subset=['nombre_oficial_norm',
↳ 'ruc'])

# 2) Preparar universo
universe = df_empresas_union_all['empresa_norm'].astype('string')
universe = universe.fillna('').map(lambda x: x.strip())
universe = universe[universe != ''].drop_duplicates().reset_index(drop=True)

choices = df_scvs_master['nombre_oficial_norm'].astype('string').tolist()
exact_set = set(choices)
n_exact = int(universe.isin(exact_set).sum())

# 3) Matching fuzzy (auto-instala rapidfuzz si falta)
engine = None
extract_one = None

try:
    from rapidfuzz import process as rf_process, fuzz as rf_fuzz
    engine = 'rapidfuzz'
    def extract_one(q: str):
        return rf_process.extractOne(q, choices, scorer=rf_fuzz.token_set_ratio)
except Exception:
    try:
        import sys, subprocess
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q',
↳ 'rapidfuzz'])
        from rapidfuzz import process as rf_process, fuzz as rf_fuzz
        engine = 'rapidfuzz'
        def extract_one(q: str):
            return rf_process.extractOne(q, choices, scorer=rf_fuzz.
↳ token_set_ratio)
    except Exception:
        engine = None
        extract_one = None

if extract_one is None:
    try:
        from thefuzz import process as fz_process, fuzz as fz_fuzz
        engine = 'thefuzz'
        def extract_one(q: str):
            return fz_process.extractOne(q, choices, scorer=fz_fuzz.
↳ token_set_ratio)
    except Exception:
        from difflib import SequenceMatcher
        engine = 'difflib'

```

```

def ratio(a: str, b: str) -> float:
    return SequenceMatcher(None, a, b).ratio() * 100
def extract_one(q: str):
    best = None
    best_score = -1.0
    for cand in choices:
        sc = ratio(q, cand)
        if sc > best_score:
            best = cand
            best_score = sc
    return (best, best_score)

# Índice para recuperar datos SCVS (incluye nombre raw + RUC)
scvs_by_norm = (df_scvs_master.sort_values(['ruc']).
    ↳drop_duplicates(subset=['nombre_oficial_norm'], keep='first')
    .set_index('nombre_oficial_norm'))

rows_match = []
for q in universe.tolist():
    res = extract_one(q)
    if res is None:
        rows_match.append({
            'empresa_norm': q,
            'scvs_nombre_norm': pd.NA,
            'scvs_nombre_raw': pd.NA,
            'status': 'no_match',
            'score': 0.0,
            'ruc': pd.NA,
        })
        continue
    if engine in ('rapidfuzz', 'thefuzz'):
        best_norm, score, *_ = res
        score = float(score)
    else:
        best_norm, score = res
        score = float(score)
    status = 'matched' if score >= MATCH_THRESHOLD else 'below_threshold'
    rec = scvs_by_norm.loc[best_norm] if best_norm in scvs_by_norm.index else_
    ↳None
    rows_match.append({
        'empresa_norm': q,
        'scvs_nombre_norm': best_norm,
        'scvs_nombre_raw': (rec['nombre_oficial_raw'] if rec is not None else_
    ↳pd.NA),
        'score': score,
        'status': status,
        'ruc': (rec['ruc'] if rec is not None else pd.NA),

```

```

    })

df_empresas_scvs_match = pd.DataFrame(rows_match)

n_total = len(df_empresas_scvs_match)
n_ok = int((df_empresas_scvs_match['status'] == 'matched').sum())
n_low = int((df_empresas_scvs_match['status'] == 'below_threshold').sum())

ruc_digits = df_empresas_scvs_match['ruc'].astype('string').fillna('')
ruc_len_13 = ruc_digits.str.len().eq(13)
n_ruc_ok = int(((df_empresas_scvs_match['status'] == 'matched') &
    ↪(df_empresas_scvs_match['ruc'].notna()))).sum())
n_ruc13_ok = int(((df_empresas_scvs_match['status'] == 'matched') & ruc_len_13).
    ↪sum())

print('[SCVS] Resumen')
print('- universo_empresas:', n_total)
print('- scvs_registros_cargados:', int(len(df_scvs_master)))
print('- exact_matches_norm:', n_exact)
print(f'- fuzzy_matches_>=_{MATCH_THRESHOLD}:', n_ok)
print(f'- fuzzy_matches_<_{MATCH_THRESHOLD}:', n_low)
print(f'- matched_con_RUC_no_nulo (>= {MATCH_THRESHOLD}):', n_ruc_ok)
print(f'- matched_con_RUC_13_digitos (>= {MATCH_THRESHOLD}):', n_ruc13_ok)
print('- motor_fuzzy:', engine)

if VERBOSE:
    display(df_scvs_master.head(5))
    display(df_empresas_scvs_match.sort_values('score', ascending=False).
    ↪head(20))

```

```

[SCVS] Resumen
- universo_empresas: 447
- scvs_registros_cargados: 219439
- exact_matches_norm: 1
- fuzzy_matches_>=_85: 146
- fuzzy_matches_<_ 85: 301
- matched_con_RUC_no_nulo (>= 85): 145
- matched_con_RUC_13_digitos (>= 85): 145
- motor_fuzzy: rapidfuzz

```

```

[20]: # --- (B2) Overrides manuales (instituciones públicas / bancos) ---
manual_mapping = {
    "CORPORACION GPF": {"ruc": "1790403755001", "nombre_oficial": "ECONOFARM S.
    ↪A."},
    "EEQ": {"ruc": "1768153530001", "nombre_oficial": "EMPRESA ELECTRICA QUITO_
    ↪S.A."},

```

```

    "ESPE": {"ruc": "1768133420001", "nombre_oficial": "UNIV. DE LAS FUERZAS_
↳ARMADAS ESPE"},
    "MINISTERIO SALUD PUBLICA": {"ruc": "1768040760001", "nombre_oficial":_
↳"MINISTERIO DE SALUD PUBLICA"},
    "MUNICIPIO DE QUITO": {"ruc": "1760001550001", "nombre_oficial": "MUNICIPIO_
↳DEL DISTRITO METROPOLITANO"},
    "CUERPO INGENIEROS EJERCITO": {"ruc": "1768001600001", "nombre_oficial":_
↳"CUERPO DE INGENIEROS DEL EJERCITO"},
    "HIDROPAUTE": {"ruc": "1768152800001", "nombre_oficial": "CELEC EP"},
    "PRESIDENCIA": {"ruc": "1768000200001", "nombre_oficial": "PRESIDENCIA DE_
↳LA REPUBLICA"},
    "BANCO DEL AUSTRO": {"ruc": "0190035042001", "nombre_oficial": "BANCO DEL_
↳AUSTRO S.A."},
    "DINERS": {"ruc": "1790151706001", "nombre_oficial": "DINERS CLUB DEL_
↳ECUADOR S.A."},
    "BANRED": {"ruc": "1791334758001", "nombre_oficial": "BANRED S.A."},
    "EL TELEGRAFO": {"ruc": "1768156200001", "nombre_oficial": "EMPRESA PUBLICA_
↳EL TELEGRAFO EP"},
}

if 'df_empresas_scvs_match' not in globals():
    raise RuntimeError('No existe df_empresas_scvs_match. Ejecuta primero la_
↳celda de matching SCVS.')

# Crear/llenar columna de verificación (qué nombre se usará como "match" final)
if 'nombre_oficial_match' not in df_empresas_scvs_match.columns:
    if 'scvs_nombre_raw' in df_empresas_scvs_match.columns:
        df_empresas_scvs_match['nombre_oficial_match'] =_
↳df_empresas_scvs_match['scvs_nombre_raw']
    else:
        df_empresas_scvs_match['nombre_oficial_match'] = pd.NA

empresa_ser = df_empresas_scvs_match['empresa_norm'].astype('string').
↳fillna('').map(lambda x: x.strip())

n_applied = 0
missing = []
for empresa_key, payload in manual_mapping.items():
    key = str(empresa_key).strip()
    mask = empresa_ser.eq(key)
    if not mask.any():
        missing.append(key)
        continue
    df_empresas_scvs_match.loc[mask, 'ruc'] = payload.get('ruc', pd.NA)
    df_empresas_scvs_match.loc[mask, 'nombre_oficial_match'] = payload.
↳get('nombre_oficial', pd.NA)

```

```

df_empresas_scvs_match.loc[mask, 'status'] = 'manual_override'
df_empresas_scvs_match.loc[mask, 'score'] = 100.0
n_applied += int(mask.sum())

print(f"[MANUAL] overrides aplicados: {n_applied} de {len(manual_mapping)} ↵
↵claves")
if missing:
    print(f"[MANUAL] claves no encontradas en empresa_norm: {len(missing)}")

```

[MANUAL] overrides aplicados: 12 de 12 claves

```

[22]: # --- (C) Export a CSV del resultado SCVS (descarga local) ---
from pathlib import Path

# Este CSV contiene el mapeo: empresa_norm -> score/status -> ruc (si se pudo ↵
↵extraer)
if 'df_empresas_scvs_match' not in globals():
    raise RuntimeError('No existe df_empresas_scvs_match. Ejecuta primero la ↵
↵celda anterior (SCVS).')

out_dir = Path('outputs')
out_dir.mkdir(parents=True, exist_ok=True)

out_path = out_dir / f"scvs_matches_threshold_{MATCH_THRESHOLD}.csv"
df_empresas_scvs_match.to_csv(out_path, index=False, encoding='utf-8-sig')

print('[EXPORT] Archivo CSV generado:', str(out_path.resolve()))
print('[EXPORT] Filas/Columnas:', df_empresas_scvs_match.shape)

```

[EXPORT] Archivo CSV generado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\01_data_ingestion_enrichment\outputs\scvs_matches_threshold_85.csv
[EXPORT] Filas/Columnas: (447, 7)